COMUNICACIONES II – 27145

PROGRAMACIÓN EN RADIO DEFINIDA POR SOFTWARE (GNURADIO)

Quezada Gómez Juan David - 2204650
Astro Bohorquez Jose David - 2201845
Gabriel Holguín Castañeda- 2212250
https://github.com/JDQG/Com2_B1_G5

1. ABSTRACT

This practice aims to provide the knowledge and skills necessary to understand and program software-defined radios (SDR) using GNURadio. It covers both theoretical concepts and their practical application, including the study of basic SDR principles, the implementation of blocks in GNURadio, and the analysis of the obtained results. For this purpose, software such as Ubuntu Linux, GNURadio, and GNURadio Companion (GRC) is used.

2. INTRODUCCIÓN

Con el avance de las tecnologías, los entornos de programación para dispositivos no solo incluyen bloques que facilitan la interacción con diversas plataformas, sino que también permiten a los usuarios desarrollar sus propios algoritmos. Esto brinda a los estudiantes la oportunidad de no solo utilizar los bloques existentes, sino también de crear nuevas funcionalidades que les permitirán afrontar desafíos más específicos, además de darles la posibilidad de trabajar con varios bloques en secuencia,con el objetivo de completar tareas más complejas, lo que ayuda al aprendizaje de los conceptos vistos en la parte teórica y complementar las capacidades de GNU Radio.

3. OBJETIVO GENERAL Y ESPECÍFICOS

1. Objetivo General: Desarrollar competencias que permitan la interacción con la programación de funciones en GNU Radio.

2. Objetivos Específicos:

a. Identificar los aspectos fundamentales en los sistemas de tiempo real y la radio definida por software.
b. Generar funciones a partir de los bloques de implementación de código y evaluar los resultados con otros bloques.
c. Utilizar los bloques implementados para producir una aplicación específica para señales reales.

4. METODOLOGÍA

Para explicar de manera clara y concisa cómo se desarrolló esta práctica de laboratorio es necesario dividir el desarrollo en 2 partes importantes, configuración de GitHub y trabajo en GNU Radio.

- **Configuración del GitHub:** Para dar comienzo a esta práctica, primero fue necesario la creación de ciertas ramas o “branches” las cuales almacenan el trabajo realizado en el laboratorio. Dentro de esta primera rama ,“Práctica 1” se crearon otras tres ramas, correspondientes a cada integrante del equipo en las cuales cada uno realiza sus aportes en el desarrollo de la práctica. Al finalizar la práctica, mediante el comando “merge” se busca unificar cada aporte realizado por cada integrante del grupo en sus respectivas ramas en la rama principal.

- **GNU Radio:** Posterior a la configuración de GitHub, se continuó con la implementación de ciertos bloques de Python en el software de GNU Radio, los cuales fueron tomados del libro guía en la sección 1.2.0.1, el cual fue proporcionado por el docente. Una vez implementados los bloques acumulador, diferenciador y el de promedio de tiempos, mediante una aplicación sobre los conceptos vistos en clase se probó su funcionalidad.

  Finalmente, mediante los comandos vistos anteriormente de GitHub, se subió a la nube las implementaciones realizadas en su carpeta correspondiente.

5. ANÁLISIS DE RESULTADOS

- **Bloque acumulador**

El bloque acumulador se utiliza para sumar valores de forma digital y consecutiva, realizando la integración de una señal respecto al tiempo. Este código implementa en GNU Radio el bloque denominado “e Acum”, que actúa como un acumulador de señales. Al heredar de “gr.sync block”, procesa un número fijo de muestras de entrada y genera una cantidad igual de muestras de salida en cada

ejecución.

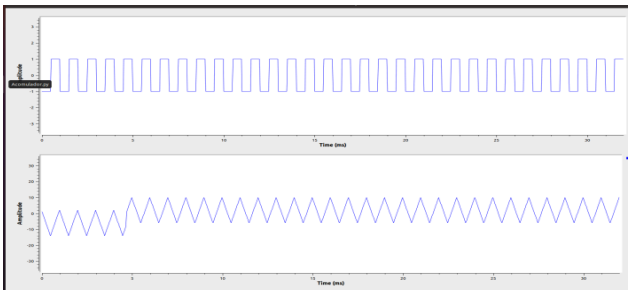

Imagen 1. Señal acumulada empleando el algoritmo del ejemplo.

Lo que se visualiza en la anterior imagen es el resultado de la implementación del algoritmo de ejemplo en el libro guía, donde a pesar de que el comportamiento de la señal resultante concuerda con lo esperado se puede apreciar una distorsión en esta, en forma de un sobresalto, esto debido a que la señal cuadrada que proviene del bloque signal source y se dirige al bloque Acumulador se trata de un flujo continuo (stream). En un instante dado, esta señal puede representarse como:

$$Xi = [x_0, x_1, \ldots, x\square, k_1]$$

Sin embargo, para procesar correctamente cada instante, es esencial tener en cuenta el valor acumulado previamente. Es decir, el bloque debe incorporar el valor que ya se ha acumulado para poder procesar el siguiente conjunto de muestras de forma consecutiva y mantener la continuidad en la suma, modificando la siguiente parte del código,

```
        self.accumulated_value = 0

    def work (self , input_items , output_items ):
        x = input_items [0]
        y0 = output_items [0]

        for i in range(len(x)):
            self.accumulated_value += x[i]
            y0[i] = self.accumulated_value

        return len (y0)
```

Imagen 2. Algoritmo corregido

En concreto, el bloque recibe señales de entrada de tipo “np.float32” y calcula la suma acumulada de dichas muestras, almacenando el resultado en la variable “self.accumulated value”. Este valor acumulado se asigna a la salida en cada iteración, de modo que la salida refleja la suma progresiva de las entradas.

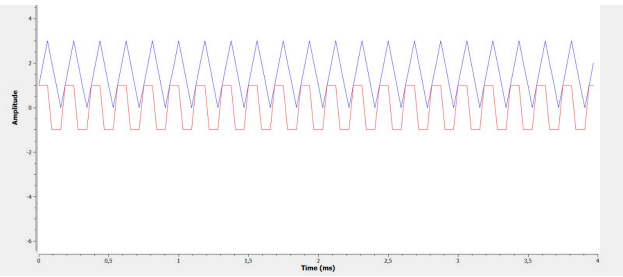

Imagen 3. Señal acumulada con la corrección.

Con la modificación implementada, se pudo depurar dicha anomalía en la gráfica resultante del bloque acumulador y se logró obtener el resultado de forma satisfactoria.

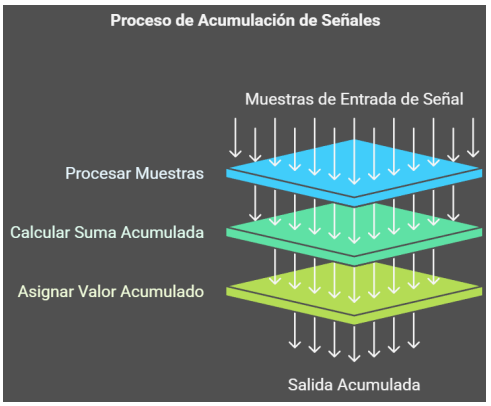

Figura 1. Esquema del algoritmo corregido.

- **Bloque diferenciador**

El bloque diferenciador se encarga de calcular la derivada discreta de la señal, es decir, . En GNURadio, se implementa mediante un bloque de Python que ejecuta un algoritmo de diferenciación. Al procesar, por ejemplo, una señal triangular y una señal cuadrada, se obtienen como resultados, respectivamente, una señal cuadrada y un tren de impulsos.

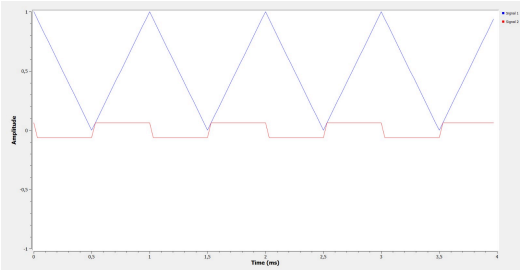

Imagen 4. Derivación señal triangular.

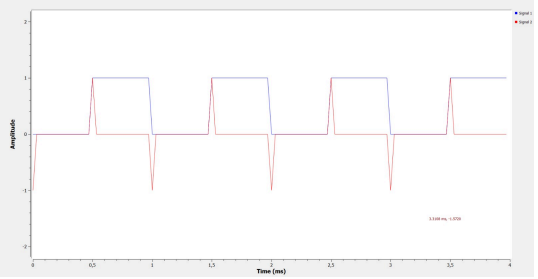

Imagen 5. Derivación señal cuadrada.

En las imágenes anteriores se puede constatar que se derivan correctamente las señales. Inicialmente, este bloque replicaba un error observado en el acumulador: la ausencia de un mecanismo para preservar el estado entre bloques de muestras, lo que generaba artefactos de discontinuidad (sobrepicos o saltos abruptos) en la señal procesada, para solucionar esto, se recurrió a lo implementado anteriormente, . El bloque calcula la derivada discreta de la señal mediante la operación $y[n] = x[n] - x[n-1]$. Para la primera muestra de cada bloque, utiliza el último valor del bloque anterior ($x[-1]$) almacenado en memoria, garantizando continuidad temporal; mientras que para las muestras subsecuentes calcula la diferencia entre cada muestra ($x[n]$) y su inmediata anterior ($x[n-1]$). Esta implementación resuelve

las discontinuidades al incorporar un mecanismo de memoria, asegurando una estimación precisa de la variación instantánea de la señal y eliminando los efectos de borde entre bloques, lo cual es crítico en aplicaciones de procesamiento en tiempo real donde la coherencia temporal es esencial. Sin embargo, observamos que para el caso de la señal triangular no alcanza la amplitud teórica esperada debido a que se utiliza la aproximación discreta mediante la diferencia entre muestras consecutivas, sin normalizar por el intervalo de muestreo. En un sistema continuo, la derivada se expresa como la variación instantánea dividida por el intervalo de tiempo; sin embargo, al calcular $y[n] = x[n] - x[n-1]$ sin aplicar dicho factor, la magnitud resultante se reduce. Así, por ejemplo, al derivar una señal triangular, las pendientes teóricas de 2 y -2 se transforman en escalones de menor amplitud, reflejando la limitación inherente de la aproximación por diferencias finitas en un entorno digital.

Figura 2. Funcionamiento del algoritmo diferenciador

- **Bloque De Aplicaciones Varias**

Respecto a los bloques proporcionados por el libro guía, estos permiten calcular métricas estadísticas clave —como la media, la media cuadrática, el valor RMS (Root Mean Square), la potencia promedio y la desviación estándar— a partir de vectores de datos. Estas herramientas resultan especialmente útiles para analizar conjuntos de información con características específicas. Por ejemplo, en aplicaciones prácticas como el estudio de la estatura de estudiantes en un aula, estos bloques facilitan el procesamiento sistemático de datos, ofreciendo una base cuantitativa para interpretar distribuciones, variabilidades y tendencias dentro de la muestra. Su implementación simplifica tareas que requieren precisión y rigor estadístico, destacando su relevancia en contextos académicos, industriales o de investigación.

El uso óptimo de los bloques programados requiere una señal continua, como una onda senoidal, cuyas propiedades matemáticas son conocidas y facilitan la validación de los resultados. Al emplear esta señal, es posible contrastar los valores calculados por los bloques (media, RMS, potencia promedio, etc.) con los teóricos, asegurando la precisión del sistema. Por ejemplo, la media de una señal seno ideal es cero, dado que sus ciclos positivos y negativos se cancelan simétricamente en el tiempo. Esta característica, verificada experimentalmente, confirma la correcta implementación de los bloques y su capacidad para procesar datos con rigor estadístico. Así, la elección de una señal de referencia predecible no solo optimiza la evaluación de los algoritmos, sino que también establece una metodología robusta para analizar sistemas en escenarios controlados.

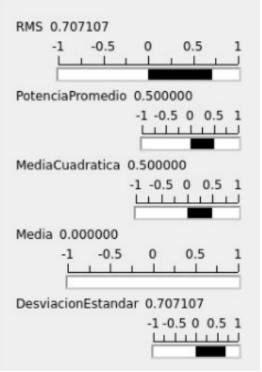

Figura 3. Valores obtenidos de la señal senoidal

6. CONCLUSIONES

- Se evidenció que el tipo de señal que sale de los bloques es importante, debido a que el tratamiento dado a un vector a una señal stream varía, por lo tanto es necesario reestructurar las señales de los bloques creados gracias al libro guía, con el fin de facilitar su tratamiento
- Se reforzó el conocimiento de programación en tiempo real, debido a que la respuesta entregada por los circuitos acumulador y diferenciador no era la esperada, para esto se modificó el código en los bloques de Python y se comprobó la respuesta con su contraparte teórica.
- La combinación de Python y GNU Radio proporciona una base robusta para el desarrollo y procesamiento de señales en tiempo real. Python destaca por su flexibilidad, mientras que GNU Radio ofrece un entorno gráfico eficiente para simular y diseñar sistemas de comunicaciones.
- La posibilidad de crear bloques personalizados en Python e integrarlos en el flujo de trabajo de GNU Radio permite implementar algoritmos avanzados y realizar ajustes dinámicos durante la ejecución.

7. REFERENCIAS

[1]"Creating your first block." [Online]. Available: https://wiki.gnuradio.org/index.php?title=

Creating_Your_First_Block

[2] "Python block with vectors." [Online]. Available: https://wiki.gnuradio.org/index.php?title=

Python_Block_with_Vectors

[3] "Comunicaciones digitales basadas en radio definida por software." [Online]. Available: https://drive.google.com/file/d/1fd9M4_

bIjwLOajQdkdN9ex2plYRoXomz/view